

Curs 1: Funcții binare

Cuprins:

- prezentarea operatorilor logici: simboluri, tabele de adevăr, relații de definire;
- prezentarea metodelor de reprezentare a funcțiilor logice:
 - tabele de adevăr complete
 - tabele de adevăr reduse
 - forme canonice
- sinteza unei scheme logice pornind de la formele canonice.

1. Introducere

O serie de procese tehnice pot fi descrise, sau cel puțin pot fi approximate, cu ajutorul propozițiilor logice bivalente. Fiecare propoziție de acest fel poate fi **adevărată** sau poate fi **falsă**. Propozițiile logice se împart în două categorii: propoziții logice simple și propoziții logice compuse.

a) Propozițiile logice simple sunt folosite de regulă pentru a indica starea unei mărimi (sau a unui parametru) ce intervine în procesul ce trebuie descris.

Exemple de propoziții logice simple: “rezervor gol”, “motor electric pornit”, “contact închis”, “presiune crescută peste limita admisă”, “temperatură sub valoarea propusă”, etc.);

b) Propozițiile logice compuse au ca sarcină descrierea completă a funcționării unui sistem. Propozițiile compuse se obțin prin combinarea, (“legarea”), în mod convenabil a mai multor propoziții logice simple. “Legarea” propozițiilor simple se face cu ajutorul operatorilor logici ce vor fi descriși mai jos.

Algebra booleană, introdusă în secolul trecut de către matematicianul englez George Boole a permis abordarea într-o manieră matematică a propozițiilor logice. În acest cadru, fiecare propoziție simplă este asimilată cu: **unu** dacă respectiva propoziție este adevărată; sau cu **zero** dacă este falsă.

Ambele tipuri de propoziții, simple sau compuse, se pot exprima prin simboluri literale la fel ca în algebra obișnuită. Spre exemplu, propoziția “contact INCHIS ” poate fi notată cu litera **x** și are valoare **x=1** dacă contactul este închis (afirmație adevărată) și respectiv **x=0** dacă contactul este deschis (afirmație falsă).

Propoziția compusă a cărei valoare logică depinde de valorile “0” sau “1” ale propozițiilor simple ce o compun, se numește **funcție logică** sau **funcție binară** deoarece, la rândul său nu poate avea decât două valori binare: 0 sau 1 .

Exemplu de propoziție compusă, deci de funcție binară:

$$\text{“contact INCHIS” } \text{ȘI} \text{ “rezervor GOL”} \Leftrightarrow F = x \bullet y$$

Semnificația notațiilor este următoarea:

- “contact INCHIS” - propoziție simplă notată cu x ;
- “rezervor GOL” - propoziție simplă notată cu y ;
- **ȘI** - conectiv (operator) logic simbolizat prin “ $\bullet$ ”;
- “contact INCHIS” **SI** “rezervor GOL” - expresia logică a propoziției compuse notată cu F ;
- $F = x \bullet y$ - expresia algebrică a funcției binare.

În exemplul considerat anterior, $F=1$ numai dacă ambele propoziții simple, x respectiv y , denumite și **variabile de intrare**, sunt adevărate adică, numai pentru cazul $x=1$ simultan cu $y=1$.

Algebra booleană este utilizată, pentru început în anul 1938 de către Shannon în elaborarea teoriei sistemelor de comutație realizate cu relee, iar mai recent a devenit fundamentul analizei și sintezei circuitelor logice și a sistemelor numerice în general.

2. Operatori logici uzuali

Așa după cum am arătat anterior, o funcție binară este alcătuită dintr-o succesiune de propoziții logice simple legate între ele prin operatori logici. În continuare, prezentăm modul de definire și simbolurile logice pentru operatorii cei mai des utilizați în descrierea funcțiilor binare. Precizăm că toate variabilele utilizate au valori în mulțimea binară $B = \{0,1\}$.

◆ Operatorul de complementare

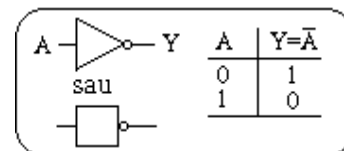
Denumiri alternative: negație logică, inversor, NOT, etc.

Expresia algebrică:

$$\text{com}(a) \stackrel{\text{def}}{=} 1 - a = \bar{a}$$

Observații:

- realizează funcția de complementare (negare) a intrării;
- prezintă o singură intrare;



◆ Operatorul de minim al intrărilor

Denumiri alternative: produs logic, ȘI logic, AND, etc.

Expresia algebrică:

$$\min(a, b, \dots, z) \stackrel{\text{def}}{=} a \cdot b \cdot \dots \cdot z$$

Expresia algebrică: $\min(a, b, \dots, z) \stackrel{\text{def}}{=} a \cdot b \cdot \dots \cdot z$

Observații:

- realizează funcția de **minim algebric** a două sau mai multe variabile binare de intrare;
- în mod curent se fabrică porți AND cu 2, 3, 4 sau 8 intrări;

Funcționare (pentru o poartă AND cu 2 intrări):

- dacă $A=0$ atunci $Y=0$, spunem că poarta este blocată;
- dacă $A=1$ atunci $Y=B$, spunem că poarta este deschisă în sensul că starea logică de pe intrarea B ajunge nemodificată la ieșire;

	A	B	Y
sau			
	A	B	Y=A·B
	0	0	0
	0	1	0
	1	0	0
	1	1	1

◆ Operatorul de maxim al intrărilor

Denumiri alternative: produal, sumă logică, SAU logic, OR, etc.

Expresia algebrică:

$$\max(a, b, \dots, z) \stackrel{\text{def}}{=} 1 - (1-a)(1-b) \dots (1-z)$$

A B Y	<table style="border-collapse: collapse; margin: 0 auto;"> <tr> <th style="padding: 2px 10px;">A</th> <th style="padding: 2px 10px;">B</th> <th style="padding: 2px 10px;">Y=A+B</th> </tr> <tr><td style="padding: 2px 10px;">0</td><td style="padding: 2px 10px;">0</td><td style="padding: 2px 10px;">0</td></tr> <tr><td style="padding: 2px 10px;">0</td><td style="padding: 2px 10px;">1</td><td style="padding: 2px 10px;">1</td></tr> <tr><td style="padding: 2px 10px;">1</td><td style="padding: 2px 10px;">0</td><td style="padding: 2px 10px;">1</td></tr> <tr><td style="padding: 2px 10px;">1</td><td style="padding: 2px 10px;">1</td><td style="padding: 2px 10px;">1</td></tr> </table>	A	B	Y=A+B	0	0	0	0	1	1	1	0	1	1	1	1
A	B	Y=A+B														
0	0	0														
0	1	1														
1	0	1														
1	1	1														
sau																

Pentru acest operator se mai utilizează o scriere simbolică de forma:

$$\max(a, b, c) \stackrel{\text{def}}{=} 1 - (1-a)(1-b)(1-c) = \left| \begin{array}{c} \text{not } a \\ b \\ c \end{array} \right|$$

Observații:

- realizează funcția de **maxim algebric** a două sau mai multe variabile binare de intrare;
- în mod curent se fabrică porți OR cu 2, 3, 4 sau 8 intrări;

Funcționare (pentru o poartă OR cu 2 intrări):

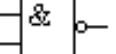
- dacă $A=0$ atunci $Y=B$, spunem că poarta este deschisă în sensul că starea logică de pe intrarea B ajunge nemodificată la ieșire (spunem că poarta este deschisă);
- dacă $A=1$ atunci $Y=1$, spunem că poarta este blocată

◆ Operatorul de minim negat al intrărilor

Denumiri alternative: ȘI-NU, NAND, etc.

Expresia algebrică:

$$\text{comin}(a, b, \dots, z) \stackrel{\text{def}}{=} 1 - \min(a, b, \dots, z) = \overline{ab\dots z}$$

A		Y	A	B	Y = $\overline{A \cdot B}$
B			0	0	1
sau			0	1	1
			1	0	1
			1	1	0

Observații:

- realizează inversul funcției logice de la punctul 2.
- funcția logică realizată de acest operator este echivalentă cu cea realizată de un ansamblu format dintr-un AND urmat de un NOT

$$\begin{array}{c} A \\ B \end{array} \rightarrow \text{NAND} \rightarrow Y \quad \Leftrightarrow \quad \begin{array}{c} A \\ B \end{array} \rightarrow \text{AND} \rightarrow R \rightarrow \text{NOT} \rightarrow Y \quad \begin{array}{l} R = A \cdot B \\ Y = \overline{R} \end{array} \Rightarrow Y = \overline{A \cdot B}$$

- în mod curent se fabrică porți NAND cu 2, 3, 4 sau 8 intrări;

Funcționare (pentru o poartă NAND cu 2 intrări):

- dacă A=0 atunci Y=1, spunem că poarta este blocată;
- dacă A=1 atunci Y= $\overline{B}$, spunem că poarta este deschisă în sensul că starea logică de pe intrarea B ajunge inversată la ieșire;

◆ Operatorul de maxim negat al intrărilor

Denumiri alternative: SAU-NU, NOR, etc.

Expresia algebrică:

$$\text{co max}(a, b, \dots, z) \stackrel{\text{def}}{=} 1 - \max(a, b, \dots, z) = \overline{\begin{array}{c} a \\ b \\ \vdots \\ z \end{array}}$$

A B Y atau ≥1	<table style="border-collapse: collapse; width: 100%;"> <tr> <th style="padding: 2px 10px;">A</th> <th style="padding: 2px 10px;">B</th> <th style="padding: 2px 10px;">Y = $\overline{A+B}$</th> </tr> <tr><td>0</td><td>0</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>0</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </table>	A	B	Y = $\overline{A+B}$	0	0	1	0	1	0	1	0	0	1	1	0
A	B	Y = $\overline{A+B}$														
0	0	1														
0	1	0														
1	0	0														
1	1	0														

Observații:

- realizează inversul funcției logice de la punctul 4.
- funcția logică realizată de acest operator este echivalentă cu cea realizată de un ansamblu format dintr-un OR urmat de un NOT

$$\begin{array}{c} A \\ B \end{array} \rightarrow \text{NOR} \rightarrow Y \quad \Leftrightarrow \quad \begin{array}{c} A \\ B \end{array} \rightarrow \text{OR} \rightarrow R \rightarrow \text{NOT} \rightarrow Y \quad \begin{array}{l} R = A + B \\ Y = \overline{R} \end{array} \Rightarrow Y = \overline{A+B}$$

- în mod curent se fabrică porți NOR cu 2, 3, 4 sau 8 intrări;

Funcționare (pentru o poartă NOR cu 2 intrări):

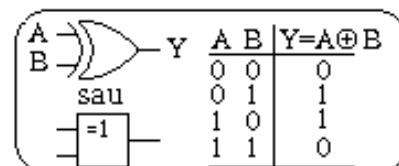
- dacă $A=0$ atunci $Y=\bar{B}$, spunem că poarta este deschisă în sensul că starea logică de pe intrarea B ajunge, inversată, la ieșire;
- dacă $A=1$ atunci $Y=0$, spunem că poarta este blocată;

◆ Operatorul de echivalență

Denumiri alternative: adunare modulo 2, SAU - EXCLUSIV, XOR, echivalență, etc.

Expresia algebrică:

$$\text{ech}(a, b) = a \oplus b \stackrel{\text{def}}{=} \begin{cases} 0 & \text{pentru } a = b \\ 1 & \text{pentru } a \neq b \end{cases}$$



Observații:

- realizează funcția de **adunare modulo 2** între două variabile binare de intrare; a două sau mai multe variabile binare de intrare;
- în mod curent se fabrică porți XOR cu 2 intrări;

Interpretarea funcționării porții XOR:

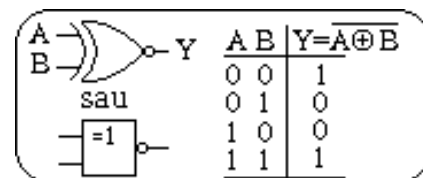
- adunare modulo 2;
- funcția de anticoincidență (ieșirea devine unu logic pentru cazurile în care intrările au stări logice diferite);
- funcție de complementare comandată:
- dacă $A=0$ atunci $Y=B$, poarta se comportă ca un repetor logic,
- variabila B trece prin poartă fără a fi negată;
- dacă $A=1$ atunci $Y=\bar{B}$, poarta se comportă ca un inversor,
- variabila B trece prin poartă și ajunge la ieșire sub formă negată.

◆ Operatorul de echivalență negat

Denumiri alternative: SAU-EXCLUSIV NEGAT, XNOR, etc.

Expresia algebrică:

$$\text{coech}(a, b) = 1 - \text{ech}(a, b) = \overline{a \oplus b} \stackrel{\text{def}}{=} \begin{cases} 1 & \text{pentru } a = b \\ 0 & \text{pentru } a \neq b \end{cases}$$



Observații:

- realizează inversul funcției de la punctul 6;
- în mod curent se fabrică porți XNOR cu 2 intrări;

Interpretarea funcționării porții XNOR:

- complement adunare modulo 2;
- funcția de coincidență (ieșirea devine unu logic dacă cele două intrări sunt conectate la aceeași stare logică)

- funcție de complementare comandată:
- dacă $A=0$ atunci $Y=\bar{B}$, poarta se comportă ca un inversor;
- dacă $A=1$ atunci $Y=B$, poarta se comportă ca un repetor logic.

3. Clasificarea circuitelor logice

În funcție de relația de legătură existentă între semnalele de intrare și cele de ieșire, relație descrisă de către funcția de transfer, circuitele logice se împart în: circuite logice combinaționale (CLC) și circuite logice secvențiale.

Circuitele logice combinaționale reprezintă clasa circuitelor logice pentru care starea ieșirilor este, în fiecare moment de timp, complet determinată numai de starea logică a intrărilor. Pentru CLC-uri nu există o dependență a ieșirilor față de stările anterioare prin care a trecut circuitul. Din acest motiv structura internă nu necesită prezența elementelor de memorare. Analiza pur logică a acestor circuite se face fără introducerea variabilei *timp*.

Circuitele logice secvențiale reprezintă clasa circuitelor logice pentru care există cel puțin un moment de timp în care ieșirile sunt determinate atât de starea prezentă a intrărilor cât și de o stare anterioară prin care a trecut circuitul. Privind lucrurile dintr-o altă perspectivă putem spune că starea prezentă a circuitului secvențial poate influența, chiar și după dispariția sa, evoluția viitoare a circuitului. Structura circuitelor secvențiale conține, în mod obligatoriu, elemente de memorare deoarece, numai prin memorare efectul unei stări prezente se poate extinde și în viitor.

Analiza circuitelor secvențiale necesită introducerea variabilei *timp* însă, aceasta este privită ca o succesiune discretă de momente $(t_1, t_2, \dots, t_i, \dots)$. Un circuit secvențial se poate descompune, la rândul său, într-un circuit pur combinațional și o zonă de memorie. Această divizare permite o abordare mai ușoară a acestor structuri.

4. Funcția de transfer a unui circuit logic combinațional

Pentru cazul general, al unui CLC cu n intrări și m ieșiri, starea semnalelor de intrare este specificată printr-un vector cu n componente $X(x_1, x_2, \dots, x_n)$, iar starea ieșirilor de un vector cu m componente $Y(y_1, y_2, \dots, y_m)$. Relația de legătură dintre intrare și ieșire este descrisă de către funcția de transfer $F(f_1, f_2, \dots, f_m)$. În cazul particular în care $m=1$, funcția de transfer are o singură componentă devenind o **funcție binară**. Studiul funcțiilor binare prezintă o importanță deosebită deoarece, conform reprezentării din fig. 1, un CLC cu m ieșiri poate fi descompus într-o sumă de m funcții binare.

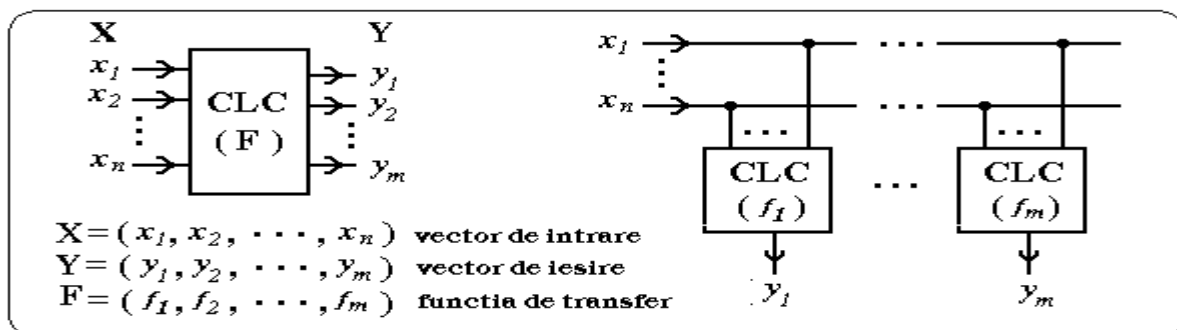


Fig. 1. Schema bloc a unui circuit logic combinațional

5. Forme de reprezentare a funcțiilor binare

Funcția de transfer a unui CLC poate fi descrisă în mai multe moduri dintre care cele mai utilizate sunt: **tabelul de adevăr**, **forma canonică** conjunctivă sau disjunctivă, **diagrama Veitch-Karnaugh**, **forma analitică redusă**, **descrierea în limbaj natural**, etc.

5. 1. Tabelul de adevăr

Tabelul de adevăr complet definește funcționarea unui sistem logic prin listarea valorilor de ieșire pentru fiecare combinație a variabilelor de intrare (vector de intrare). În cazul general al unui CLC cu n intrări și m ieșiri tabelul de adevăr va conține:

- 2^n linii, unde 2 la puterea n reprezintă numărul maxim al combinațiilor de intrare;
- $n+m$ coloane, câte una pentru fiecare variabilă de intrare și respectiv, componentă a funcției de transfer).

Spre exemplu, în fig. 2, se prezintă tabelul de adevăr complet pentru un CLC caracterizat de 3 intrări și 2 ieșiri. Pentru acest caz, numărul de linii ale tabelului de adevăr este 8 iar numărul de coloane 5. Tot aici, se prezintă modul de descompunere a funcției de transfer în cele două funcții binare ce o alcătuiesc.

Tabelele de adevăr ale unor funcții binare de mai multe variabile devin complexe atunci când numărul variabilelor de intrare crește. Pentru a simplifica scrierea, se poate utiliza **tabelul de adevăr redus**, în care se trec numai combinațiile de intrare pentru care funcția are aceeași valoare logică. Apar astfel două forme ale tabelului redus: una pentru valorile "adevărat" și alta pentru valorile "fals" ale funcției. Modul de trecere de la o formă la alta a tabelului de adevăr este ilustrat în fig. 3.

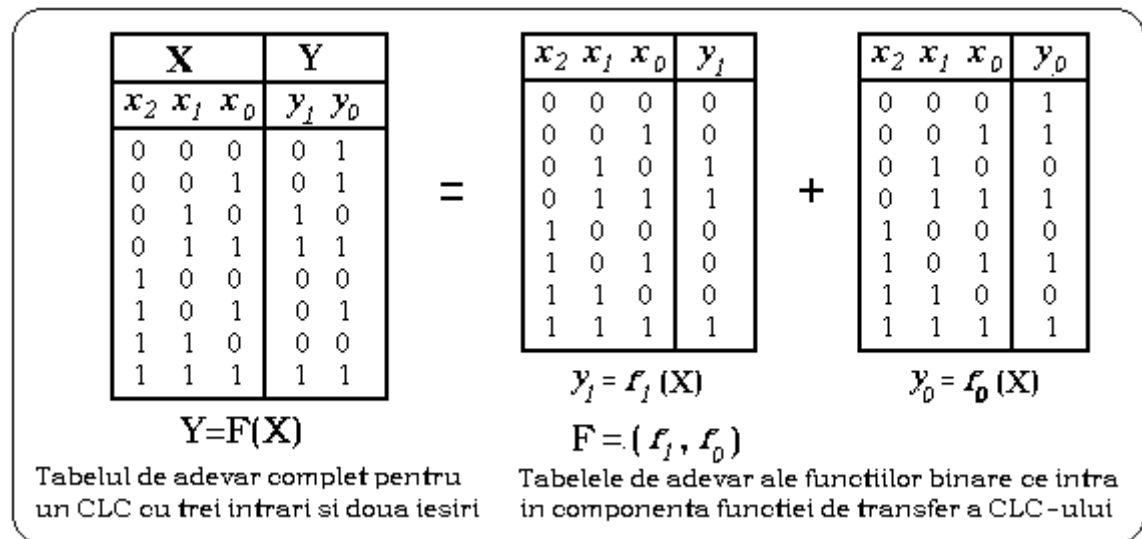


Fig. 2 Tabelul de adevăr complet al unui un CLC cu 3 intrări și 2 ieșiri

Tabelul de adevăr complet al unei funcții binare de 3 variabile

x_2	x_1	x_0	y_1
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Tabelul de adevăr redus pentru valorile de zero logic

x_2	x_1	x_0	y_1
0	0	*	0
1	0	*	0
1	1	0	0

x_2	x_1	x_0	y_1
*	0	*	0
1	1	0	0

Tabelul de adevăr redus pentru valorile de unu logic

x_2	x_1	x_0	y_0
0	1	*	1
1	1	1	1

* = don't care (poate fi "zero" sau "unu" logic)

Fig. 3. Trecerea de la forma completă la formele reduse ale tabelor de adevăr

5. 2. Formele canonice ale unei funcții binare

Forma canonică constituie o altă modalitate de reprezentare a funcțiilor binare în care, descrierea funcționării se face analitic. Scrierea canonică a unei funcții binare se poate face în două moduri:

♦ prin utilizarea unei sume a tuturor mintermenilor pentru care funcția binară prezintă valoarea "1" - cazul primei forme canonice, denumită și **formă canonică disjunctivă**;

♦ prin utilizarea unui produs al tuturor maxtermenilor pentru care funcția binară prezintă valoarea "0"- cazul celei de-a doua forme canonice denumită și **formă canonică conjunctivă**.

Definiții:

• **Mintermenul este un produs logic la care participă fiecare variabilă de intrare o singură dată: sub formă directă - dacă variabila de intrare este unu logic, sau sub formă complementată - dacă variabila de intrare este zero logic.**

• **Maxtermenul este o sumă logică, un produal, la care participă fiecare variabilă de intrare luată o singură dată: sub formă directă - dacă variabila de intrare este zero logic, sau sub formă complementată - dacă variabila de intrare este unu logic.**

Exemplu:

Considerăm funcția de transfer prezentată în figura 2.

• **Modul de definire al mintermenilor și al maxtermenilor** se prezintă în tabelul de mai jos:

Linie	Vector de intrare X	MINTERMEN	MAXTERMEN	Vector de ieșire Y
	$x_2 \ x_1 \ x_0$			$y_1 \ y_0$
0	0 0 0	$m_0 = \bar{x}_2 \cdot \bar{x}_1 \cdot \bar{x}_0$	$M_0 = x_2 + x_1 + x_0$	0 1
1	0 0 1	$m_1 = \bar{x}_2 \cdot \bar{x}_1 \cdot x_0$	$M_1 = x_2 + x_1 + \bar{x}_0$	0 1
2	0 1 0	$m_2 = \bar{x}_2 \cdot x_1 \cdot \bar{x}_0$	$M_2 = x_2 + \bar{x}_1 + x_0$	1 0
3	0 1 1	$m_3 = \bar{x}_2 \cdot x_1 \cdot x_0$	$M_3 = x_2 + \bar{x}_1 + \bar{x}_0$	1 1
4	1 0 0	$m_4 = x_2 \cdot \bar{x}_1 \cdot \bar{x}_0$	$M_4 = \bar{x}_2 + x_1 + x_0$	0 0
5	1 0 1	$m_5 = x_2 \cdot \bar{x}_1 \cdot x_0$	$M_5 = \bar{x}_2 + x_1 + \bar{x}_0$	0 1
6	1 1 0	$m_6 = x_2 \cdot x_1 \cdot \bar{x}_0$	$M_6 = \bar{x}_2 + \bar{x}_1 + x_0$	0 0
7	1 1 1	$m_7 = x_2 \cdot x_1 \cdot x_0$	$M_7 = \bar{x}_2 + \bar{x}_1 + \bar{x}_0$	1 1

• **Prima formă canonică** (forma disjunctivă) a celor două funcții binare:

$$y_1 = m_2 + m_3 + m_7 = \bar{x}_2 x_1 \bar{x}_0 + \bar{x}_2 x_1 x_0 + x_2 x_1 x_0 = \begin{vmatrix} \bar{x}_2 x_1 \bar{x}_0 \\ \bar{x}_2 x_1 x_0 \\ x_2 x_1 x_0 \end{vmatrix}$$

$$y_0 = m_0 + m_1 + m_3 + m_5 + m_7 = \bar{x}_2 \bar{x}_1 \bar{x}_0 + \bar{x}_2 \bar{x}_1 x_0 + \bar{x}_2 x_1 x_0 + x_2 \bar{x}_1 x_0 + x_2 x_1 x_0 = \begin{vmatrix} \bar{x}_2 \bar{x}_1 \bar{x}_0 \\ \bar{x}_2 \bar{x}_1 x_0 \\ \bar{x}_2 x_1 x_0 \\ x_2 \bar{x}_1 x_0 \\ x_2 x_1 x_0 \end{vmatrix}$$

• **A doua formă canonică** (forma conjunctivă) a celor două funcții binare:

$$y_1 = M_0 M_1 M_4 M_5 M_6 = (x_2 + x_1 + x_0)(x_2 + x_1 + \bar{x}_0)(\bar{x}_2 + x_1 + x_0)(\bar{x}_2 + x_1 + \bar{x}_0)(\bar{x}_2 + \bar{x}_1 + x_0)$$

$$y_1 = M_0 M_1 M_4 M_5 M_6 = \begin{vmatrix} x_2 & x_2 & \bar{x}_2 & \bar{x}_2 & \bar{x}_2 \\ x_1 & x_1 & x_1 & x_1 & \bar{x}_1 \\ x_0 & \bar{x}_0 & x_0 & \bar{x}_0 & x_0 \end{vmatrix}$$

$$y_0 = M_2 M_4 M_6 = (x_2 + \bar{x}_1 + x_0)(\bar{x}_2 + x_1 + x_0)(\bar{x}_2 + \bar{x}_1 + x_0) = \begin{vmatrix} x_2 & \bar{x}_2 & \bar{x}_2 \\ \bar{x}_1 & x_1 & \bar{x}_1 \\ x_0 & x_0 & x_0 \end{vmatrix}$$

Observații:

- Se remarcă faptul că atât mintermenii, cât și maxtermenii, sunt dependenți numai de starea logică a variabilelor de intrare și nu depind de valorile de ieșire ale funcției logice.
- Din analiza expresiilor celor două forme canonice se observă că, deși descriu aceleași funcție logică, cu cât o formă canonică este mai restrânsă cu atât cealaltă este mai amplă.
- În practică se recomandă utilizarea primei forme canonice pentru funcțiile binare care au un număr mai redus de valori "adevarat", printre valorile funcției, și a formei a doua în caz contrar.
- Pentru exemplul considerat, se obțin expresii mai simple dacă se utilizează prima formă canonică pentru y_2 , respectiv cea de-a doua formă.

6. Trasarea schemei logice pentru formele canonice

Modul de trasare a schemelor logice pornind de la formele canonice se prezintă pentru un circuit combinațional cu trei intrări specificat prin tabelul de adevăr de mai jos:

x_2	x_1	x_0	y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

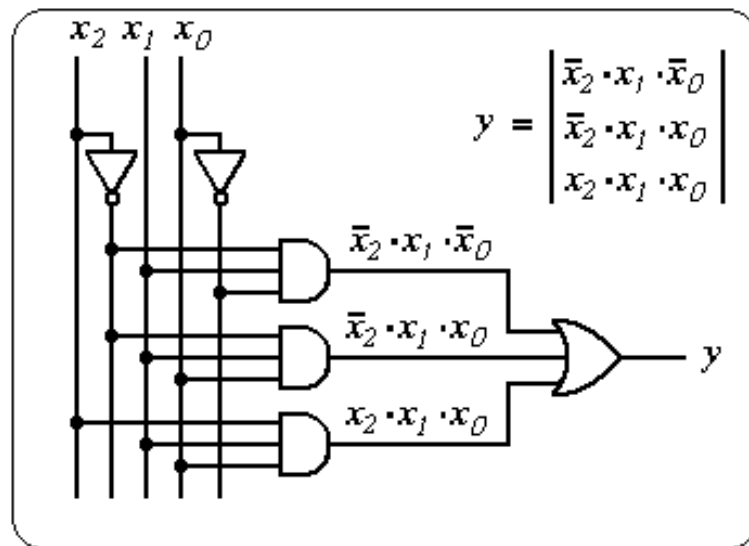
Prima formă canonică, denumita adesea **formă disjunctivă**, se obține luând în considerație doar valorile de *unu logic* pe care le are funcția ce trebuie implementată. Mai

concret, prima formă canonică este o sumă logică în care trebuie luați toți mintermenii care corespund valorilor de unu logic de pe coloana de ieșire a tabelului de adevăr.

În cazul de față, prima formă canonică are expresia:

$$y = m_2 + m_3 + m_7 = \begin{vmatrix} m_2 \\ m_3 \\ m_7 \end{vmatrix} = \begin{vmatrix} \bar{x}_2 x_1 \bar{x}_0 \\ \bar{x}_2 x_1 x_0 \\ x_2 x_1 x_0 \end{vmatrix}$$

Pentru implementare, fiecare mintermen este calculat cu o poartă AND cu trei intrări, iar sumarea mintermenilor se face cu o poartă OR, așa cum se poate vedea în figura de mai jos:

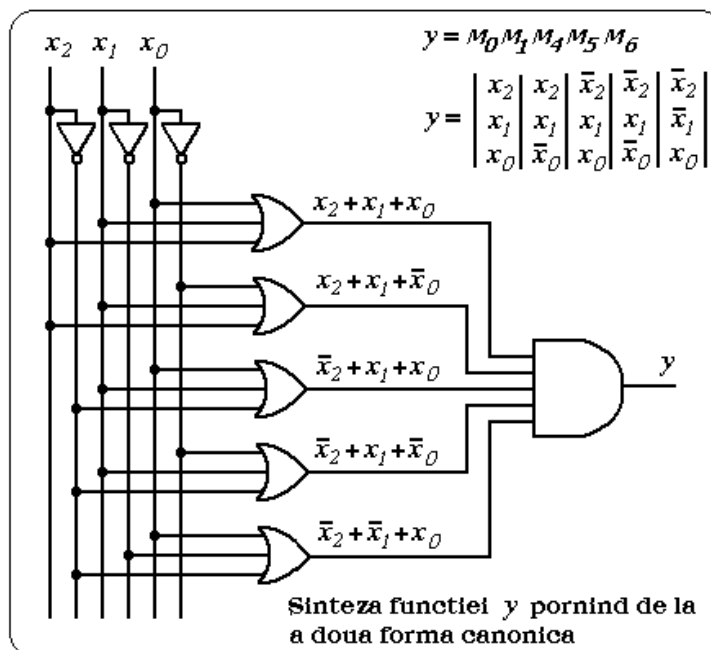


A doua formă canonică, denumita adesea **formă conjunctivă**, se obține luând în considerație doar valorile de zero logic pe care le are funcția. Mai concret, a doua formă canonică este un produs logic în care trebuie luați toți maxtermenii care corespund valorilor de zero logic de pe coloana de ieșire a tabelului de adevăr.

În cazul de față, a doua formă are expresia:

$$y = M_0 M_1 M_4 M_5 M_6 = \begin{vmatrix} x_2 & x_2 & \bar{x}_2 & \bar{x}_2 & \bar{x}_2 \\ x_1 & x_1 & x_1 & x_1 & \bar{x}_1 \\ x_0 & \bar{x}_0 & x_0 & \bar{x}_0 & x_0 \end{vmatrix} = \begin{vmatrix} x_2 & x_2 & \bar{x}_2 & \bar{x}_2 & \bar{x}_2 \\ x_1 & x_1 & x_1 & x_1 & \bar{x}_1 \\ x_0 & \bar{x}_0 & x_0 & \bar{x}_0 & x_0 \end{vmatrix}$$

Pentru implementare, fiecare maxtermen necesită o poartă OR cu trei intrări, iar produsul logic se realizează cu o poartă AND cu număr convenabil de intrări. Modul de conectare se poate vedea în figura de mai jos:

**Observații:**

- formele canonice nu sunt minimale, ele se mai pot simplifica folosind proprietățile algebrei binare;
- de regulă, sinteza funcțiilor logice nu se face după formele canonice deoarece rezultă scheme logice prea complexe;
- în mod normal, sinteza se face după expresiile algebrice simplificate pentru a reduce necesarul de porți logice;
- cu cât o formă canonică are o expresie mai amplă cu atât cealaltă formă are o scriere mai redusă;

7. Teoremele lui DeMorgan

Teoremele lui DeMorgan ne oferă o metodă rapidă de transformare a sumelor logice în produse, respectiv a produselor în sume logice.

Pentru cazul unor sume/produse cu trei variabile, formele standard pentru teoremele lui DeMorgan se scriu astfel:

$$\overline{xyz} = \bar{x} + \bar{y} + \bar{z}$$

$$\overline{x + y + z} = \bar{x} \cdot \bar{y} \cdot \bar{z}$$

Pe lângă formele standard, în practică sunt foarte utile și următoarele forme:

$$xyz = \overline{\bar{x} + \bar{y} + \bar{z}}$$

$$x + y + z = \overline{\overline{x} \cdot \overline{y} \cdot \overline{z}}$$

Schemele logice rezultate din teoremele lui DeMorgan sunt prezentate în tabelul de mai jos.

Teorema lui DeMorgan	Reprezentarea grafică
$\overline{xyz} = \overline{x} + \overline{y} + \overline{z}$	
$xyz = \overline{\overline{x} + \overline{y} + \overline{z}}$	
$\overline{x + y + z} = \overline{x} \cdot \overline{y} \cdot \overline{z}$	
$x + y + z = \overline{\overline{x} \cdot \overline{y} \cdot \overline{z}}$	

Analizând schemele logice din tabelul anterior se constată că trecerea de la o schemă la alta se face dacă realizăm simultan trei acțiuni:

- se complementează variabilele de intrare;
- se înlocuiește operatorul de minim (AND) cu cel de maxim (OR), respectiv operatorul de maxim cu cel de minim;
- se complementează variabila de ieșire.

Acțiunile de mai sus pot fi aplicate în transformarea schemelor realizate în logică combinată în scheme ce folosesc logica de același tip.